Universidad Carlos III

Sistemas Distribuidos

Curso 2024-25

Ejercicio Evaluable 2

Miembros:

Javier Díez 100495833@alumnos.uc3m.es

Mario Hidalgo 100495801@alumnos.uc3m.es

Tabla de contenido

1. Resumen de diseño..... 3
Plan de pruebas.....4
2. Compilación y generación del ejecutable.....5

1. Resumen de diseño

La implementación del servidor y del proxy mediante sockets TCP introduce un cambio significativo en la forma técnica en la que se realiza la comunicación entre cliente y servidor respecto al mecanismo inicial basado en colas de mensajes POSIX. En esta nueva implementación, se utilizan las funciones estándar del API de sockets TCP tales como socket(), bind(), listen(), accept(), send() y recv(), encargadas de gestionar la comunicación de red directa entre los componentes del sistema.

El servidor (servidor-sock.c) establece inicialmente un socket mediante la función socket() utilizando la familia AF_INET y el protocolo SOCK_STREAM para conexiones TCP fiables. Este socket se asocia posteriormente a un puerto específico mediante la función bind(). Para escuchar solicitudes entrantes, se invoca a la función listen(), tras lo cual, mediante un bucle infinito, se ejecuta continuamente la función accept() para aceptar conexiones entrantes de clientes. Cada conexión aceptada genera un nuevo hilo utilizando la función pthread_create, garantizando la concurrencia al permitir múltiples solicitudes simultáneas.

Además, en el servidor se implementaron funciones específicas como string_to_request() y response_to_string(), cuya finalidad es convertir estructuras internas de datos a cadenas formateadas claramente mediante delimitadores específicos. Esta transformación es crucial para mantener independencia del lenguaje y simplificar la depuración. Se emplea además un mutex (pthread_mutex_t) para proteger secciones críticas del servidor, particularmente durante las operaciones de recepción y procesamiento de solicitudes.

El proxy (proxy-sock.c) también realiza cambios técnicos relevantes al sustituir completamente las colas de mensajes POSIX por conexiones TCP directas al servidor. El proxy crea un socket TCP con la función socket() y establece conexión mediante connect() al servidor usando parámetros configurables globalmente (dirección IP y puerto). Estas configuraciones pueden establecerse a través de variables de entorno, proporcionando flexibilidad operativa.

La comunicación proxy-servidor está protegida mediante un mutex para evitar condiciones de carrera en escenarios con múltiples hilos concurrentes. Al igual que en el servidor, se implementan las mismas funciones de conversión

(request_to_string() y string_to_response()) asegurando la coherencia del protocolo de comunicación.

Estos cambios técnicos han permitido reemplazar un esquema específico basado en colas de mensajes POSIX por un mecanismo más general y estandarizado basado en sockets TCP, lo cual mejora notablemente la escalabilidad, robustez, interoperabilidad y facilita la integración futura con clientes desarrollados en otros lenguajes y tecnologías.

Plan de pruebas

A parte de las pruebas ya implementadas para la anterior entrega, hemos añadido unas pocas pruebas más.Finalmente hemos desarrollado el archivo el archivo app-cliente.c. Este código contiene 11 casos de pruebas distintas para comprobar el correcto funcionamiento del proyecto.

ID	Caso	Descripción	Llamada cliente	Mensaje recibido	Mensaje esperado
2	-	Insertar una clave con set_value con valores double	set_value(keys[i], v1, 3.0, v2, v3)	set_value() para clave 3 retornó: 0 (esperado 0) set_value() para clave 4 retornó: 0 (esperado 0)	set_value() para clave 3 retornó: 0 (esperado 0) set_value() para clave 4 retornó: 0 (esperado 0)
5	1	Modificar un valor de una clave existente con valores double	modify_value(4, new_v1, 2.0, new_v2, new_v3)	modify_value() para clave 4 retornó: 0 (esperado 0)	modify_value() para clave 4 retornó: 0 (esperado 0)
10	1	Verificar límites N_value2 con un valor double y siendo un único elemento	set_value(60, "Un solo elemento", 1, single_val, v3);	set_value() para clave 60 con N_value2=1 retornó: 0 (esperado 0)	set_value() para clave 60 con N_value2=1 retornó: 0 (esperado 0)

La siguiente tabla refleja las pruebas que surgen cuando se ejecutan los comandos en terminal

ID	Descripción	Línea de comando	Mensaje recibido	Mensaje esperado	Notas adicionales
1	El puerto introducido está ocupado	./servidor 8080	bind: Address already in use Error: no se pudo enlazar al puerto 8080. Intentando con un puerto dinámico… Puerto asignado dinámicamente: <puerto> Servidor de sockets iniciado. Escuchando en el puerto <puerto>...	bind: Address already in use Error: no se pudo enlazar al puerto 8080. Intentando con un puerto dinámico… Puerto asignado dinámicamente: <puerto> Servidor de sockets iniciado. Escuchando en el puerto <puerto>...	Es necesario que PORT_TUPLAS sea igual al puerto devuelto por programa para que el programa funcione correctamente (cuando se vaya a ejecutar el cliente).
2	No se puede conectar a la IP del cliente	env IP_TUPLAS=126.0.0.3 PORT_TUPLAS=8085 ./cliente	connect: Connection timed out Error: No se pudo conectar al servidor en 126.0.0.3:8085	connect: Connection timed out Error: No se pudo conectar al servidor en 126.0.0.3:8085	Hay que esperar un timer para obtener el mensaje de error.
3	El puerto del cliente no coincide con el del servidor	env IP_TUPLAS=localhost PORT_TUPLAS=8086 ./cliente	connect: Connection refused Error: No se pudo conectar al servidor en localhost:8086	connect: Connection refused Error: No se pudo conectar al servidor en localhost:8086	-

2. Compilación y generación del ejecutable

El makefile tiene las siguientes funciones:

- make: Compila la biblioteca dinámica libclaves.so, el cliente app-cliente y el servidor servidor-sock.
- make servidor: Compila el servidor.
- make cliente: Compila el cliente.
- make clean: Limpia los archivos generados durante la compilación.